

FORMATION EMBARQUÉE

TP1 — Seance 4

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Fiche de séance 4 : portage ESP32 et MQTT (4 h)

En-tête pédagogique

Objectifs	Porter un projet multi-modules sur une autre carte ; se connecter en Wi-Fi ; publier en MQTT avec reconnexion propre ; vérifier de bout en bout
Prérequis	Séances 1-3 finies ; module 03 §7 étape 6
Matériel	ESP32 DevKit + DHT22 + OLED (ou projet Wokwi « ESP32 ») ; sur PC : <code>mosquitto-clients</code> ou le client web HiveMQ
Livrable	La station publie ses mesures en JSON toutes les 10 s sur un broker public, et survit aux coupures réseau

Déroulé minuté

0:00-0:40 — Portage matériel et logiciel

Nouveau câblage (3,3 V ! le DHT22 accepte, vérifie ton OLED) :

Élément	Uno (avant)	ESP32 (après)
DHT22 DATA	D2	GPIO 4
OLED SDA/SCL	A4/A5	GPIO 21/22 (I2C par défaut)
LED alerte	D8	GPIO 2 (LED carte sur beaucoup de modules)
Bouton	D3	GPIO 15
Potentiomètre	A0	GPIO 34 (entrée seule, ADC1)

Dans l'IDE : installer le support ESP32 (gestionnaire de cartes), carte « ESP32 Dev Module ». Adaptations de code attendues **uniquement** dans les constantes de broches des modules (c'est le test de l'architecture de la séance 2) + `anaLogRead` qui renvoie 0..4095 (12 bits) → adapter la mise à l'échelle de la consigne.

✅ **Point de contrôle 1 (0:40)** : la station de la séance 3 tourne à l'identique sur ESP32. Note dans ton journal les 4-5 lignes qui ont changé — si tu as dû toucher `capteur.cpp` au-delà du numéro de broche, remonte la cause.

0:40-1:30 — Module réseau : Wi-Fi + MQTT

Nouveau module `reseau.h/.cpp` (bibliothèque `PubSubClient`) :

```
// reseau.h
void reseau_init(const char* ssid, const char* mdp,
                const char* broker, const char* id_client);
void reseau_tick(uint32_t maintenant_ms);    // entretient la connexion
bool reseau_connecte();
bool reseau_publier(const char* topic, const char* json);
```

```
// reseau.cpp – les points délicats
#include "reseau.h"
#include <WiFi.h>
#include <PubSubClient.h>

static WiFiClient net;
static PubSubClient mqtt(net);
static const char *ssid_, *mdp_, *id_;
static uint32_t derniere_tentative = 0;

void reseau_init(const char* ssid, const char* mdp,
                const char* broker, const char* id_client) {
    ssid_ = ssid; mdp_ = mdp; id_ = id_client;
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid_, mdp_);           // NON bloquant : on n'attend pas ici
    mqtt.setServer(broker, 1883);
}

void reseau_tick(uint32_t now) {
    if (WiFi.status() != WL_CONNECTED) return;    // le Wi-Fi retente seul

    if (!mqtt.connected()) {
        // Reconnexion au plus 1 fois / 5 s : JAMAIS en rafale
        if (now - derniere_tentative >= 5000) {
            derniere_tentative = now;
            mqtt.connect(id_);
        }
        return;
    }
    mqtt.loop();           // entretien : à appeler très souvent
}

bool reseau_connecte() { return WiFi.status() == WL_CONNECTED && mqtt.connected(); }

bool reseau_publier(const char* topic, const char* json) {
    return reseau_connecte() && mqtt.publish(topic, json);
}
```

Points de conception à retenir : - **Aucune attente bloquante** : pas de `while (WiFi.status() != ...)` — la station mesure et affiche MÊME sans réseau (le réseau est un service, pas une condition de vie). - Reconnexion **espacée** (5 s) : un broker public bannit les clients qui martèlent. - **id_client unique** (mets ton prénom + un nombre) : deux clients de même id se déconnectent mutuellement en boucle — LE grand classique des brokers publics.

1:30-2:15 — Publication JSON

Dans le `.ino`, nouvelle tâche périodique (10 s) :

```
if (now - t_publie >= 10000 && !capteur_en_panne()) {
    t_publie = now;
    char json[96];
    snprintf(json, sizeof json,
             "{\"t\":%.1f,\"h\":%.1f,\"alerte\":%s}",
             capteur_temperature(), capteur_humidite(),
             alarme ? "true" : "false");
    reseau_publier("formation/TONPRENOM/meteo", json);
}
```

Ajoute l'état réseau sur l'OLED (icône ou « WiFi/MQTT/-- »).

Wokwi : SSID `Wokwi-GUEST`, mot de passe vide ; broker `broker.hivemq.com`.

2:15-3:00 — Vérification de bout en bout

Sur PC :

```
mosquitto_sub -h broker.hivemq.com -t "formation/TONPRENOM/#" -v
```

(ou le client web <http://www.hivemq.com/demos/websocket-client/>).

✓ **Point de contrôle 2** : les JSON arrivent toutes les 10 s. Puis les trois pannes à provoquer et documenter :

Panne provoquée	Comportement attendu
Coupure Wi-Fi (désactive le routeur / bouton Wokwi)	station vivante, OLED indique « -- », reprise SEULE au retour
Capteur débranché	plus de publication (on ne publie pas du NAN), écran « CAPTEUR HS », le reste vit
Broker inaccessible (mauvais nom 2 min)	tentatives espacées de 5 s visibles, pas de gel

3:00-3:45 — Consommer les données (aperçu du module 05)

Petit consommateur en Python sur PC (10 lignes, `pip install paho-mqtt`) :

```
import json, paho.mqtt.client as mqtt

def sur_message(cli, _, msg):
    m = json.loads(msg.payload)
    print(f"{msg.topic} : {m['t']} °C, {m['h']} %"
          + (" *** ALERTE ***" if m.get("alerte") else ""))

cli = mqtt.Client()
cli.on_message = sur_message
cli.connect("broker.hivemq.com", 1883)
cli.subscribe("formation/TONPRENOM/meteo")
cli.loop_forever()
```

Tu viens de construire une chaîne IoT complète : capteur → firmware C++ → Wi-Fi → broker → application PC. (La version Java robuste est l'objet du mini-projet du module 05.)

3:45-4:00 — Commit final + auto-évaluation

```
git add . && git commit -m "TP1 seance 4 : ESP32 + MQTT avec reconnexion"
```

Remplis la grille /20 de la fiche TP principale ([tp1-arduino-station-meteo.md](#) §grille), **avec preuves** (captures du moniteur, de `mosquitto_sub`, du journal Git).

Erreurs fréquentes

Symptôme	Cause	Remède
Déconnexions MQTT en boucle	id client non unique sur broker public	id avec prénom + aléa
<code>Brownout detector triggered</code> (ESP32 redémarre)	alimentation USB faiblarde au pic Wi-Fi	autre câble/port, condensateur 470 µF
OLED muette après portage	OLED 5 V sur bus 3,3 V, ou mauvais pins I2C	vérifier module et GPIO 21/22
<code>analogRead</code> plafonne bizarrement	GPIO 34-39 = entrée seule OK, mais ADC2 inutilisable avec Wi-Fi	rester sur ADC1 (GPIO 32-39)
Publications qui cessent après des heures	tas fragmenté par des <code>String</code>	<code>snprintf</code> + tampons fixes (déjà le cas ici)

Pour aller plus loin (hors séance)

Trois extensions, chacune = un commit de portfolio : **OTA** (mise à jour par Wi-Fi, bibliothèque ArduinoOTA) ; **deep sleep** entre mesures (autonomie batterie ×50, mais il faut repenser l'OLED et le bouton — bon exercice de conception) ; **Node-RED** sur PC pour tracer les courbes et déclencher un e-mail d'alerte.